MLA02 – FUNDAMENTALS OF MACHINE LEARNING
QUESTION 5

K-Nearest Neighbours (K-NN) Classification

Introduction

K-NN assigns a class based on the majority class among the nearest training samples. Feature scaling is applied before distance-based classification.

Implementation library / technique: StandardScaler + KNeighborsClassifier(n_neighbors=3).

General workflow: prepare the data, separate features and target labels, train the model, classify a new sample, and display the result.

Problem 1: Iris Flower Classification

Aim

To implement K-Nearest Neighbours (K-NN) Classification for iris flower classification and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes SepalLength, SepalWidth, PetalLength, PetalWidth and their class labels to train a model, then classify a new sample.

Algorithm

1. Import the required Python libraries.
2. Store the training samples in a feature matrix X and labels in y.
3. Create the classifier and fit it using X and y.
4. Prepare the new sample and apply the same preprocessing, if required.
5. Predict and display the class of the new sample.

Input (Training Data)

SepalLength	SepalWidth	PetalLength	PetalWidth	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
5.0	3.4	1.5	0.2	0
6.0	2.7	4.5	1.5	1
5.9	3.0	4.2	1.5	1
6.1	2.8	4.7	1.2	1
6.5	3.0	5.5	1.8	2
6.7	3.1	5.6	2.4	2
6.3	2.9	5.4	1.9	2

Program

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth']
training_data = [[5.1, 3.5, 1.4, 0.2, 0], [4.9, 3.0, 1.4, 0.2, 0], [5.0, 3.4, 1.5, 0.2, 0], [6.0, 2.7, 4.5, 1.5, 1], [5.9, 3.0, 4.2, 1.5, 1], [6.1, 2.8, 4.7, 1.2, 1], [6.5, 3.0, 5.5, 1.8, 2], [6.7, 3.1, 5.6, 2.4, 2], [6.3, 2.9, 5.4, 1.9, 2]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)
new_sample = np.array([5.2, 3.4, 1.5, 0.2], dtype=float)
```

```
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, K-Nearest Neighbours (K-NN) Classification was implemented for iris flower classification and the new sample was classified using the trained model.

Problem 2: Fruit Classification

Aim

To implement K-Nearest Neighbours (K-NN) Classification for fruit classification and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Weight, Sweetness, ColorCode and their class labels to train a model, then classify a new sample.

Algorithm

6. Import the required Python libraries.
7. Store the training samples in a feature matrix X and labels in y.
8. Create the classifier and fit it using X and y.
9. Prepare the new sample and apply the same preprocessing, if required.
10. Predict and display the class of the new sample.

Input (Training Data)

Weight	Sweetness	ColorCode	Target
120	8	1	0
130	9	1	0
150	7	2	1
160	8	2	1
180	6	3	2
200	5	3	2
125	8	1	0
155	7	2	1
190	5	3	2

Program

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['Weight', 'Sweetness', 'ColorCode']
training_data = [[120, 8, 1, 0], [130, 9, 1, 0], [150, 7, 2, 1], [160, 8, 2, 1], [180, 6, 3, 2], [200, 5, 3, 2],
[125, 8, 1, 0], [155, 7, 2, 1], [190, 5, 3, 2]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)
new_sample = np.array([145, 8, 2], dtype=float)
```

```
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, K-Nearest Neighbours (K-NN) Classification was implemented for fruit classification and the new sample was classified using the trained model.

Problem 3: Pass/Fail Prediction

Aim

To implement K-Nearest Neighbours (K-NN) Classification for pass/fail prediction and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes StudyHours, Attendance, AssignmentScore and their class labels to train a model, then classify a new sample.

Algorithm

11. Import the required Python libraries.
12. Store the training samples in a feature matrix X and labels in y.
13. Create the classifier and fit it using X and y.
14. Prepare the new sample and apply the same preprocessing, if required.
15. Predict and display the class of the new sample.

Input (Training Data)

StudyHours	Attendance	AssignmentScore	Target
1	50	30	0
2	55	40	0
3	60	55	0
4	70	65	1
5	75	70	1
6	80	85	1
2	45	35	0
7	90	90	1
3	65	50	0

Program

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['StudyHours', 'Attendance', 'AssignmentScore']
training_data = [[1, 50, 30, 0], [2, 55, 40, 0], [3, 60, 55, 0], [4, 70, 65, 1], [5, 75, 70, 1], [6, 80, 85, 1],
[2, 45, 35, 0], [7, 90, 90, 1], [3, 65, 50, 0]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)
new_sample = np.array([5, 78, 75], dtype=float)
```

```
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, K-Nearest Neighbours (K-NN) Classification was implemented for pass/fail prediction and the new sample was classified using the trained model.

Problem 4: Vehicle Type Classification

Aim

To implement K-Nearest Neighbours (K-NN) Classification for vehicle type classification and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Weight, EngineCC, Wheels and their class labels to train a model, then classify a new sample.

Algorithm

16. Import the required Python libraries.
17. Store the training samples in a feature matrix X and labels in y.
18. Create the classifier and fit it using X and y.
19. Prepare the new sample and apply the same preprocessing, if required.
20. Predict and display the class of the new sample.

Input (Training Data)

Weight	EngineCC	Wheels	Target
900	800	4	0
1000	1000	4	0
1200	1200	4	0
1800	1600	4	1
2000	2000	4	1
2200	2500	4	1
500	100	2	2
600	125	2	2
700	150	2	2

Program

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['Weight', 'EngineCC', 'Wheels']
training_data = [[900, 800, 4, 0], [1000, 1000, 4, 0], [1200, 1200, 4, 0], [1800, 1600, 4, 1], [2000,
2000, 4, 1], [2200, 2500, 4, 1], [500, 100, 2, 2], [600, 125, 2, 2], [700, 150, 2, 2]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)
new_sample = np.array([1100, 1000, 4], dtype=float)
```

```
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, K-Nearest Neighbours (K-NN) Classification was implemented for vehicle type classification and the new sample was classified using the trained model.

Problem 5: Customer Segment Classification

Aim

To implement K-Nearest Neighbours (K-NN) Classification for customer segment classification and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Age, AnnualSpend, VisitsPerMonth and their class labels to train a model, then classify a new sample.

Algorithm

21. Import the required Python libraries.
22. Store the training samples in a feature matrix X and labels in y.
23. Create the classifier and fit it using X and y.
24. Prepare the new sample and apply the same preprocessing, if required.
25. Predict and display the class of the new sample.

Input (Training Data)

Age	AnnualSpend	VisitsPerMonth	Target
22	12000	2	0
25	15000	3	0
28	18000	4	0
35	45000	8	1
40	50000	9	1
45	60000	10	1
52	25000	3	2
58	28000	4	2
62	30000	3	2

Program

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['Age', 'AnnualSpend', 'VisitsPerMonth']
training_data = [[22, 12000, 2, 0], [25, 15000, 3, 0], [28, 18000, 4, 0], [35, 45000, 8, 1], [40, 50000,
9, 1], [45, 60000, 10, 1], [52, 25000, 3, 2], [58, 28000, 4, 2], [62, 30000, 3, 2]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)
new_sample = np.array([38, 48000, 8], dtype=float)
```

```
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, K-Nearest Neighbours (K-NN) Classification was implemented for customer segment classification and the new sample was classified using the trained model.

Conclusion

Five different application problems were demonstrated using K-Nearest Neighbours (K-NN) Classification. The results show how a trained machine-learning model can use example data to classify a new observation. Performance depends on the quality and size of the training data.

Requirements

Python 3, scikit-learn, NumPy, and a Python IDE or Jupyter Notebook.